

Antojarte

Estructura de carpetas y guía de inicialización

Backend (Nest.js) + Frontend (React / Vite)

1. Estructura de carpetas — Backend (Nest.js)

```
antojarte-api/
├── src/
│   ├── main.ts
│   ├── app.module.ts
│   ├── config/
│   │   ├── database.config.ts
│   │   ├── common/
│   │   │   ├── guards/
│   │   │   │   ├── jwt-auth.guard.ts
│   │   │   │   ├── optional-jwt-auth.guard.ts
│   │   │   ├── decorators/
│   │   │   │   ├── current-user.decorator.ts
│   │   │   ├── filters/
│   │   │   │   ├── http-exception.filter.ts
│   │   ├── auth/ (module, controller, service, strategies/, dto/)
│   │   ├── users/ (entities/user.entity.ts, module, controller, service, dto/)
│   │   ├── addresses/ (entities/address.entity.ts, module, controller, service, dto/)
│   │   ├── products/ (entities/{product,product-variant,product-image}.entity.ts, ...)
│   │   ├── tags/ (entities/{tag,product-tag}.entity.ts, module, controller, service)
│   │   ├── reviews/ (entities/review.entity.ts, module, controller, service, dto/)
│   │   ├── cart/ (entities/{cart,card-item}.entity.ts, module, controller, service, dto/)
│   │   ├── coupons/ (entities/coupon.entity.ts, module, service – SIN controller)
│   │   ├── orders/ (entities/{order,order-item}.entity.ts, module, controller, service, dto/)
│   │   ├── payments/ (entities/payment.entity.ts, module, controller, service)
│   ├── .env
│   ├── .env.example
│   ├── package.json
│   └── tsconfig.json
```

Notar que **coupons/** no tiene controller.ts: no expone rutas propias, solo el service que consumen cart y orders.

2. Estructura de carpetas — Frontend (React + Vite)

```
antojarte-web/
├── src/
│   ├── main.jsx
│   ├── App.jsx
│   ├── router/routes.jsx
│   ├── styles/index.css      (import de Tailwind v4 + design tokens)
│   ├── assets/              (fuentes, imágenes, modelos .glb)
│   ├── components/
│   │   ├── ui/              (Button, Card, Badge, Input)
│   │   ├── layout/          (Navbar, Footer)
│   │   ├── home/            (Hero3D, CategoryChips, FeaturedGrid)
│   │   ├── product/         (ProductCard, ProductGallery, ProductFilters)
│   │   ├── cart/            (CartDrawer, CartItem)
│   │   ├── checkout/        (StepIndicator, ShippingForm, OrderSummary)
│   │   ├── auth/            (AuthModal, LoginForm, RegisterForm)
│   │   └── pages/
│   │       ├── Home.jsx
│   │       ├── Shop.jsx
│   │       ├── ProductDetail.jsx
│   │       ├── Checkout.jsx
│   │       ├── OrderConfirmation.jsx
│   │       └── Account.jsx
│   ├── store/               (cartStore.js, authStore.js – Zustand)
│   ├── services/            (api.js, products/cart/orders/auth.service.js)
│   ├── three/               (CookieModel.jsx, HeroScene.jsx)
│   ├── animations/          (useScrollHero.js, gsapConfig.js)
│   └── utils/               (formatCurrency.js)
├── public/
├── index.html
├── vite.config.js
└── package.json
```

3. Guía paso a paso

Prerrequisitos

Se usa pnpm en vez de npm: instala las dependencias de forma estricta (solo se puede importar lo que está declarado en package.json, sin phantom dependencies) y comparte un store común entre proyectos, ahorrando espacio en disco.

- Node.js 20 LTS o superior (node -v para verificar)
- pnpm como gestor de paquetes (npm install -g pnpm, o corepack enable pnpm)
- MySQL instalado localmente, o corriendo vía Docker
- Editor recomendado: VS Code

Backend — Nest.js

```
# 1. Instalar el CLI de Nest globalmente
pnpm add -g @nestjs/cli

# 2. Crear el proyecto (forzando pnpm como gestor de paquetes)
nest new antojarte-api --package-manager pnpm
cd antojarte-api

# 3. Base de datos (TypeORM + driver de MySQL)
pnpm add @nestjs/typeorm typeorm mysql2

# 4. Variables de entorno
pnpm add @nestjs/config

# 5. Autenticación (JWT)
pnpm add @nestjs/jwt @nestjs/passport passport passport-jwt
pnpm add -D @types/passport-jwt

# 6. Hash de contraseñas
pnpm add bcrypt
pnpm add -D @types/bcrypt

# 7. Validación de DTOs
pnpm add class-validator class-transformer

# 8. DTOs parciales (PATCH) y guest_token
pnpm add @nestjs/mapped-types uuid
pnpm add -D @types/uuid

# 9. Documentación automática de la API (recomendado)
pnpm add @nestjs/swagger

# 10. Aprobar scripts de instalación nativos (bcrypt los necesita)
pnpm approve-builds
```

.env del backend:

```
DB_HOST=localhost
DB_PORT=3306
DB_USERNAME=root
DB_PASSWORD=tu_password
DB_DATABASE=antojarte
JWT_SECRET=un_secreto_largo_y_random
JWT_EXPIRES_IN=7d
```

Generar los módulos con el CLI:

```
nest g module products && nest g controller products && nest g service products
nest g module tags && nest g controller tags && nest g service tags
nest g module cart && nest g controller cart && nest g service cart
nest g module coupons && nest g service coupons
nest g module orders && nest g controller orders && nest g service orders
nest g module payments && nest g controller payments && nest g service payments
nest g module reviews && nest g controller reviews && nest g service reviews
nest g module users && nest g controller users && nest g service users
nest g module addresses && nest g controller addresses && nest g service addresses
nest g module auth && nest g controller auth && nest g service auth
```

```
# Levantar el servidor en modo desarrollo (hot-reload)
pnpm run start:dev
```

Con `TypeOrmModule.forRootAsync()` en `app.module.ts` (leyendo el `.env` vía `ConfigService`) se conecta todo a MySQL. Con `synchronize: true` en desarrollo, TypeORM crea las tablas automáticamente a partir de las entidades — no hace falta escribir el `CREATE TABLE` a mano, solo definir las clases `@Entity()` siguiendo el DER ya definido.

Frontend — React + Vite

```
# 1. Crear el proyecto (variante JavaScript, no TypeScript)
pnpm create vite antojarte-web --template react
cd antojarte-web
pnpm install

# 2. Tailwind CSS v4 (se integra como plugin de Vite)
pnpm add tailwindcss @tailwindcss/vite

# 3. Ruteo
pnpm add react-router-dom

# 4. Animaciones
pnpm add gsap

# 5. 3D — Three.js + helpers de React
pnpm add three @react-three/fiber @react-three/drei

# 6. Estado global (carrito, auth)
pnpm add zustand

# 7. Cliente HTTP
pnpm add axios

# 8. Íconos (Lucide, definido en los design tokens)
pnpm add lucide-react
```

vite.config.js:

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import tailwindcss from '@tailwindcss/vite'

export default defineConfig({
  plugins: [react(), tailwindcss()],
})
```

src/styles/index.css:

```
@import "tailwindcss";

@theme {
  --color-text-main: #2E2D2C;
  --color-surface: #F7F5FF;
  --color-bg: #FFFBEB;
  --color-accent-secondary: #FFD580;
  --color-accent-main: #8D572E;
  --color-accent-tertiary: #A889A;
  --font-display: "Playfair Display", serif;
  --font-sans: "Lato", sans-serif;
}
```

.env del frontend:

```
VITE_API_URL=http://localhost:3000

# Levantar el servidor de desarrollo
pnpm run dev
```

Con esto ambos proyectos quedan corriendo en paralelo: Nest.js normalmente en el puerto 3000, Vite en el 5173, listos para empezar a llenar con las entidades del DER y los componentes de los wireframes.